Université de Sherbrooke
Faculté des sciences – Département d'informatique

IFT605/IFT713 – Systèmes répartis et multiagents
Automne 2026 – Projet 1

SmartAgent-IoT

Plateforme multi-agents de domotique
MQTT, openHAB et Java RMI

Rapport de projet

Équipe :
Amévi Solange ETSE
Cheikh Ahmadou Tidiane LY
Mame Tabara SY
Matricules : [matricules à compléter]

Résumé. Ce rapport décrit la conception, l'installation, la validation et l'analyse critique d'une plateforme qui supervise trois pièces d'un bâtiment (`salleA`, `salleB`, `cuisine`). Des capteurs simulés publient leurs mesures sur un broker MQTT ; un agent Java par pièce les mémorise et les expose par Java RMI ; un coordinateur rééquilibre la température lorsque l'écart entre deux pièces voisines dépasse 4 °C ; openHAB affiche l'ensemble et applique des règles d'automatisation. Chaque affirmation de ce rapport est appuyée par une sortie réelle du système (captures d'écran et extraits de journaux).

27 octobre 2026

Table des matières

1 Introduction 1
2 Architecture 1
3 Mise en place de l'environnement 2
4 MQTT : topics, QoS et simulateur 3
5 openHAB : Things, Items et règles 4
6 Java RMI : agents et registre 5
7 Coordinateur : politique d'équilibrage 6
8 Tolérance aux pannes 7
9 Tests et validation 9
10 Analyse critique 9
11 Difficultés rencontrées 10
12 Conclusion 10
A Reproduire le système (PowerShell) 11

1 Introduction

Contexte. Une plateforme de domotique repose sur des composants hétérogènes (capteurs, actionneurs, logiciel de supervision) qui doivent communiquer de façon fiable, même lorsque l'un d'eux tombe en panne. Le projet demande de réaliser une telle plateforme pour un étage de bâtiment à trois pièces, en combinant trois technologies vues en cours : la messagerie publish/subscribe **MQTT**, la plateforme d'automatisation **openHAB** et les appels de méthodes distantes **Java RMI**.

Objectifs.

— simuler les capteurs de chaque pièce (température, luminosité, occupation) toutes les 5 secondes ;
— associer un agent Java à chaque pièce, accessible à distance par RMI sur le port 2199 ;
— configurer openHAB (Bridge, Things, Items, règles DSL, tableau de bord) ;
— rééquilibrer la température entre pièces voisines si l'écart dépasse 4 °C ;
— tolérer les pannes : capteur muet (timeout de 30 s) et agent arrêté (`RemoteException`).

Démarche. Le travail s'est déroulé en cinq étapes : (1) mise en place de l'environnement sous Windows 11 (Docker, openHAB, Maven) ; (2) compilation des trois modules et lancement des six programmes Java ; (3) vérification du flux de données de bout en bout ; (4) ajout d'un échange direct entre agents (`consulterVoisin`) après relecture de l'énoncé et du barème ; (5) validation systématique des scénarios, y compris les deux types de pannes. Les sections 3 à 8 détaillent ces points ; la section 10 discute honnêtement les limites de la solution.

2 Architecture

2.1 Vue d'ensemble

La figure 1 présente les huit éléments du système : deux conteneurs Docker (Mosquitto et openHAB) et six programmes Java (un registre RMI, trois agents, un simulateur, un coordinateur). Les flux MQTT (en haut) transportent les mesures et les consignes ; les appels RMI (au centre) relient le coordinateur aux agents ; le coordinateur informe openHAB par l'API REST.

Composant	Exécution	Rôle
Mosquitto 2	conteneur, port 1883	Broker MQTT (accès anonyme, environnement de test).
openHAB 4.3.0	conteneur, port 8080	Tableau de bord, règles, API REST ; partage le réseau de Mosquitto.
Registre RMI	JVM dédiée, port 2199	Annuaire des agents (`agent-salleA`, `agent-salleB`, `agent-cuisine`).
Agents (×3)	une JVM par pièce	Abonné MQTT + serveur RMI ; mémorise les dernières mesures.
Simulateur	une JVM, 3 threads	Publie les trois mesures de chaque pièce toutes les 5 s.
Coordinateur	une JVM	Un cycle d'équilibrage toutes les 5 s, publication de l'état vers openHAB.

TABLE 1 – Composants et processus.

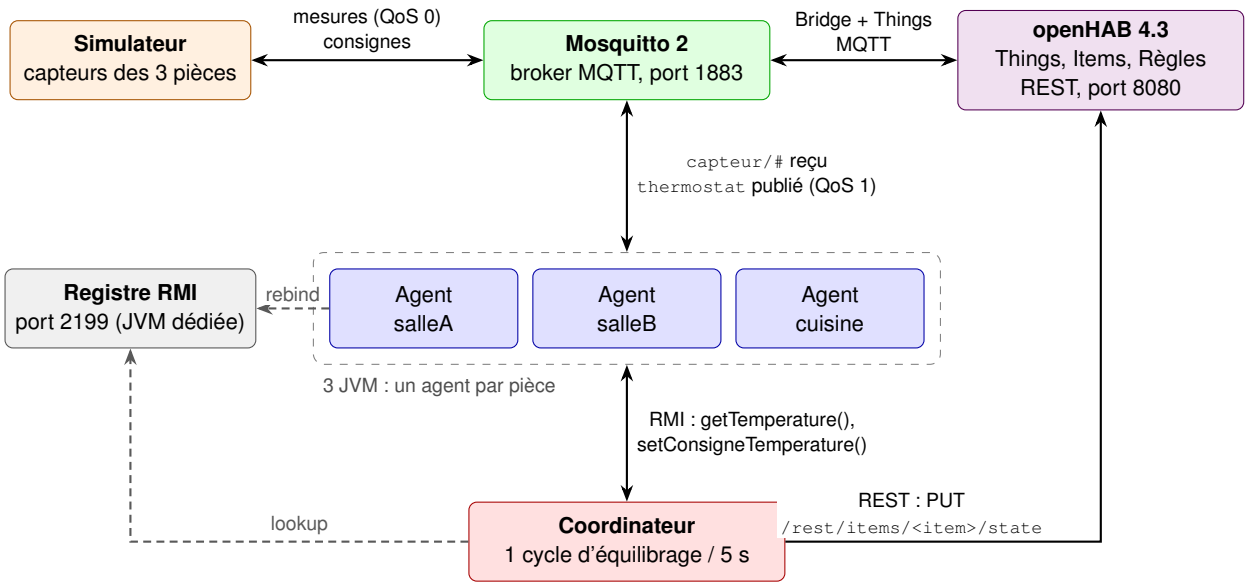


FIGURE 1 – Architecture de SmartAgent-IoT : composants, protocoles et ports.

2.2 Organisation du code et choix de conception

Le dépôt est un projet Maven multi-modules (Java 21, packages `ca.usherbrooke.ift605.*`) avec trois modules : `sensor-simulator`, `agent-piece` et `coordinateur`. Le coordinateur dépend de `agent-piece` pour réutiliser l'interface distante et les constantes partagées ; le plugin *shade* produit un jar exécutable autonome par module (`*-all.jar`). Quatre décisions structurent la solution :

— **une JVM par agent** : arrêter un agent (panne simulée) n'affecte jamais les autres ;
— **registre RMI dans son propre processus** : s'il était hébergé par le premier agent, arrêter cet agent ferait disparaître l'annuaire pour tout le monde ;
— **logique séparée des entrées/sorties** : `EtatPiece` (mesures et timeout, horloge injectable) et les interfaces `FournisseurAgents` / `NotificateurOpenHab` permettent de tester le coordinateur avec de faux agents, sans réseau ;
— **constantes centralisées** (`Constantes`) : port, noms RMI, convention des topics, timeout.

3 Mise en place de l'environnement

Le développement s'est fait sous Windows 11 avec PowerShell. Outils et versions : JDK Oracle 21.0.12, Apache Maven 3.10.0 (plugins compiler 3.13.0, surefire 3.2.5, shade 3.5.3), Eclipse Paho 1.2.5, JUnit Jupiter 5.10.2, Docker Desktop (WSL 2), images `eclipse-mosquitto:2` et `openhab/openhab:4.3.0`.

Docker et conteneurs. Docker Desktop exige la fonctionnalité Windows *Virtual Machine Platform* : au premier lancement, il a affiché « Virtual Machine Platform not enabled ». Il a fallu l'activer, mettre à jour WSL (`wsl --update`) et redémarrer l'ordinateur. Le fichier `docker-compose.yml` démarre ensuite les deux services ; la directive `network_mode: "service:mosquitto"` fait partager à openHAB la pile réseau du conteneur Mosquitto, de sorte que, vu d'openHAB, le broker est simplement `localhost:1883`. Les dossiers `conf-openhab/` (Things, Items, règles, sitemap) et `openhab-data/` sont montés dans le conteneur, si bien que la configuration est versionnée avec le code. La figure 2 montre les deux services actifs (openHAB « healthy »).

```
PS C:\Users\Administrator\Desktop\UDS\Automne\IFT 605  IFT 713 - Systèmes répartis et multiagents\Projets\Projet1\smartagent-iot> doc
ker compose ps
NAME                  IMAGE                     COMMAND                  SERVICE     CREATED       STATUS                   PORTS
smartagent-mosquitto   eclipse-mosquitto:2     "/docker-entrypoint.…"   mosquitto   2 weeks ago   Up 22 minutes            0.0.0.0:1
883->1883/tcp, [::]:1883->1883/tcp, 0.0.0.0:8080->8080/tcp, [::]:8080->8080/tcp
smartagent-openhab     openhab/openhab:4.3.0   "/entrypoint gosu op…"   openhab     2 weeks ago   Up 22 minutes (healthy)
PS C:\Users\Administrator\Desktop\UDS\Automne\IFT 605  IFT 713 - Systèmes répartis et multiagents\Projets\Projet1\smartagent-iot>
```

FIGURE 2 – `docker compose ps` : Mosquitto et openHAB 4.3.0 actifs.

Premier démarrage d'openHAB. L'assistant a demandé la création d'un compte administrateur, puis l'installation de modules (persistance MapDB et RRD4j, interface Basic UI). Le **MQTT Binding** n'est pas installé par

défaut : il a été ajouté via « Magasin de modules » (bouton *REMOVE* visible une fois installé). Après chargement de nos fichiers, les quatre *Things* (le Bridge et les trois pièces) sont passés **ONLINE** (figure 3).

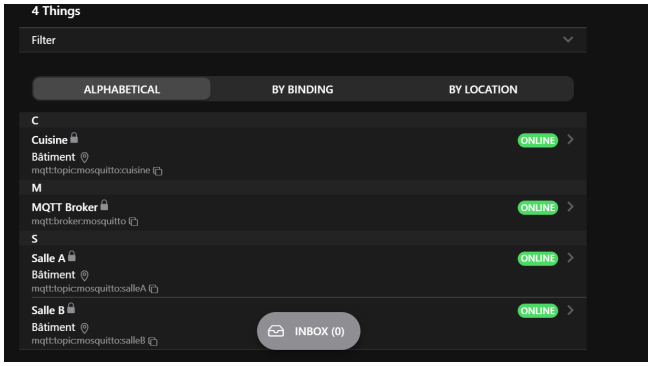


FIGURE 3 – Les quatre Things en ligne : Bridge `mqtt:broker:mosquitto` et trois pièces.

Compilation et lancement. La commande `mvn clean package` compile les trois modules, exécute les 15 tests JUnit et produit les jars (figure 4). Les avertissements du plugin *shade* (fichier `META-INF/MANIFEST.MF` présent dans plusieurs jars) sont sans conséquence. Les scripts `.sh` fournis ne s'exécutent pas sous PowerShell : nous avons donc écrit un bloc PowerShell équivalent (annexe A) qui lance les six programmes en arrière-plan, journaux redirigés vers `logs/`. Windows crée un processus *lanceur* par programme : six programmes donnent donc **douze** processus `java.exe` (figure 10), et un arrêt propre exige `taskkill /T` pour tuer l'arbre complet.

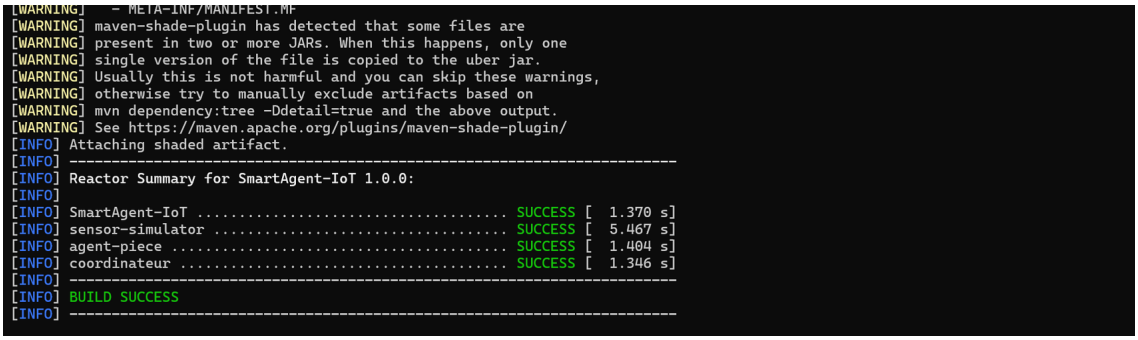


FIGURE 4 – Fin de `mvn clean package` : *Reactor Summary* et `BUILD SUCCESS` (15 tests, 0 échec).

4 MQTT : topics, QoS et simulateur

Convention de topics. Tous les topics suivent `batiment/<piece>/<famille>/<nom>` (tableau 2). Les agents s'abonnent à `batiment/<piece>/capteur/#` uniquement : chaque agent ne voit que sa pièce.

Topic	Publié par	QoS	Contenu
`batiment/<piece>/capteur/temperature`	simulateur	0	décimal, 15 à 30 °C
`batiment/<piece>/capteur/luminosite`	simulateur	0	entier, 0 à 1000 lux
`batiment/<piece>/capteur/occupation`	simulateur	0	`true` / `false`
`batiment/<piece>/actuateur/thermostat`	agent (RMI), openHAB	1 (agent)	consigne en °C
`batiment/<piece>/simulateur/<cmd>`	manuel (démo)	0	`temperature`, `occupation`, `panne`

TABLE 2 – Topics MQTT du système.

Choix de QoS. Les mesures sont publiées en **QoS 0** (« au plus une fois ») : une mesure perdue est remplacée 5 secondes plus tard, donc l'acquittement coûterait plus qu'il ne rapporte. La consigne de thermostat est une *commande* : elle est publiée en **QoS 1** (« au moins une fois ») pour ne pas être perdue ; la consigne étant

idempotente, un doublon éventuel est inoffensif. Le QoS 2 aurait ajouté un échange à quatre messages sans bénéfice ici.

Robustesse du client. Agents et simulateur utilisent Eclipse Paho avec reconnexion automatique. Comme la session est `cleanSession=true`, le broker oublie les abonnements à la coupure : la méthode `connectComplete` se réabonne donc après chaque reconnexion.

Simulateur contrôlable. Il publie trois mesures par pièce toutes les 5 secondes (température bornée à 15–30 °C, luminosité 0–1000 lux, occupation qui bascule avec 5 % de chance par cycle). Pour rendre la démonstration reproductible, il écoute aussi : la *consigne* (la température dérive de 25 % par cycle vers elle, ce qui simule un chauffage) et les topics `simulateur/*` (forcer une température ou une occupation, `panne=true` pour couper les publications d'une pièce). Cette boucle de régulation est visible dans les journaux : les températures convergent vers la consigne de leur pièce.

5 openHAB : Things, Items et règles

Bridge et Things. Dans openHAB 4, le broker est un *Bridge* et chaque pièce un *Thing* placé dans ce Bridge. Chaque Thing déclare quatre canaux (température, luminosité, occupation, thermostat), le thermostat ayant un `commandTopic` pour que openHAB *publie* les consignes qu'il envoie.

```
Bridge mqtt:broker:mosquitto [ host="localhost", port=1883, secure=false ]
{
    Thing topic salleA "Salle A" @ "Bâtiment" {
        Channels:
            Type number : temperature [ stateTopic="batiment/salleA/capteur/temperature" ]
            Type switch : occupation  [ stateTopic="batiment/salleA/capteur/occupation", on="true", off="false" ]
            Type number : thermostat  [ stateTopic="batiment/salleA/actuateur/thermostat",
                                        commandTopic="batiment/salleA/actuateur/thermostat" ]
    }   // ... idem pour salleB et cuisine
}
```

Items et fonction *expire*. Les Items sont liés aux canaux. Les trois mesures portent `expire="30s,state=UNDEF"` : sans mise à jour pendant 30 s, l'état passe à `UNDEF`. C'est une fonction du cœur d'openHAB (aucun add-on) qui répond à l'exigence « UNDEF si le capteur ne répond plus ».

```
Number SalleA_Temperature "Température Salle A [%.1f]" <temperature> (gTemperatures)
       { channel="mqtt:topic:mosquitto:salleA:temperature", expire="30s,state=UNDEF" }
```

Règles DSL. Neuf règles sont définies (tableau 3). Les règles d'économie et de confort exploitent le fait que `sendCommand` sur l'Item thermostat *publie* la consigne sur MQTT.

Règle (nombre)	Déclencheur	Action
Économie d'énergie (×3)	occupation `changed to OFF`	consigne abaissée à 18 °C
Confort (×3)	occupation `changed to ON`	consigne remontée à 21 °C
Alerte surchauffe	un membre de `gTemperatures` change	si > 28 °C : `Alerte_Batiment`
Capteur hors ligne	un membre de `gTemperatures` passe à `UNDEF`	`Alerte_Batiment` + avertissement au journal
Équilibrage signalé	`Batiment_DernierEquilibrage received update`	copie dans `Alerte_Batiment` + journal

TABLE 3 – Les neuf règles openHAB.

Tableau de bord. Le *sitemap* `batiment` présente, pour chaque pièce, les trois mesures et un réglage de consigne (*Setpoint*, 15 à 30 °C), plus un cadre « Système » (état des agents, dernier équilibrage, alerte). La figure 5 montre le système en fonctionnement.

Preuve que les règles s'exécutent. Le journal d'openHAB (fichier `openhab.log` dans le conteneur, qui n'apparaît pas dans `docker logs`) contient les règles d'économie et de confort déclenchées par les changements d'occupation, ainsi que la règle d'équilibrage (figure 6). Les horodatages d'openHAB sont en UTC (4 heures d'avance sur l'heure locale, UTC−4).

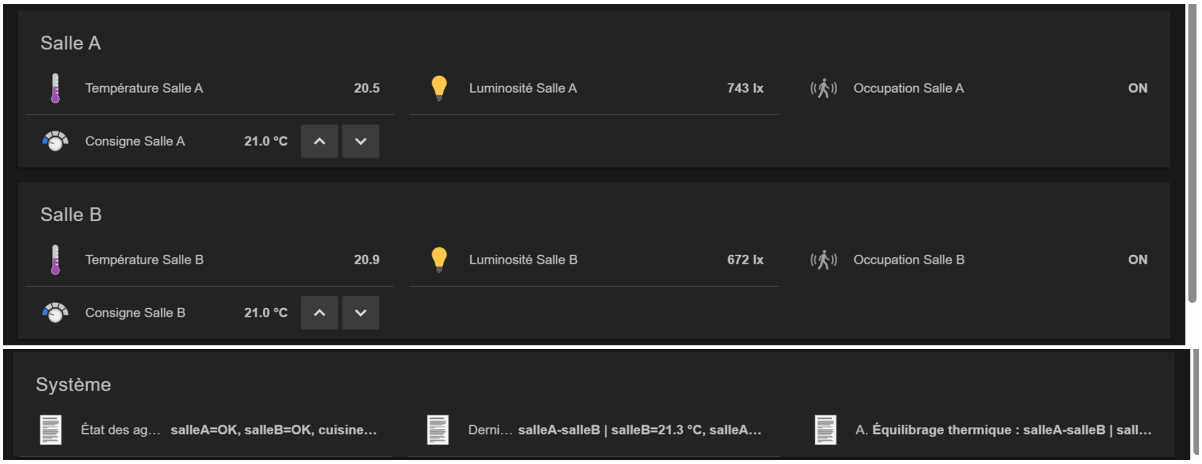


FIGURE 5 – Tableau de bord openHAB (Basic UI) : mesures et consignes de Salle A et Salle B (haut), puis cadre « Système » alimenté par le coordinateur (bas : état des agents, dernier équilibrage, alerte).

```
PS C:\Users\Administrator\Desktop\UDS\Automne\IFT 605  IFT 713 - Systèmes répartis et multiagents\Projets\Projet1\smartagent-iot> doc
ker compose exec openhab sh -c "grep -i -E 'consigne|alerte' /openhab/userdata/logs/openhab.log | tail -n 15"
2026-10-27 16:37:06.031 [INFO ] [g.openhab.core.model.script.batiment] - Salle B vide : consigne abaissée à 18 °C
2026-10-27 16:38:01.241 [INFO ] [g.openhab.core.model.script.batiment] - Cuisine vide : consigne abaissée à 18 °C
2026-10-27 16:38:26.159 [INFO ] [g.openhab.core.model.script.batiment] - Salle B occupée : consigne remontée à 21 °C
2026-10-27 16:38:31.289 [INFO ] [g.openhab.core.model.script.batiment] - Cuisine occupée : consigne remontée à 21 °C
2026-10-27 16:38:36.169 [INFO ] [g.openhab.core.model.script.batiment] - Salle B vide : consigne abaissée à 18 °C
2026-10-27 16:39:36.064 [INFO ] [g.openhab.core.model.script.batiment] - Salle A occupée : consigne remontée à 21 °C
2026-10-27 16:39:46.822 [INFO ] [g.openhab.core.model.script.batiment] - Cuisine vide : consigne abaissée à 18 °C
2026-10-27 16:40:16.875 [INFO ] [g.openhab.core.model.script.batiment] - Cuisine occupée : consigne remontée à 21 °C
2026-10-27 16:41:24.161 [INFO ] [g.openhab.core.model.script.batiment] - Le coordinateur a rééquilibré : salleB-cuisine | cuisine=21.
6 °C, salleB=17.5 °C | écart 4.1 > 4.0 -> consigne 19.6 envoyée à cuisine
2026-10-27 16:41:32.027 [INFO ] [g.openhab.core.model.script.batiment] - Cuisine vide : consigne abaissée à 18 °C
2026-10-27 16:41:47.059 [INFO ] [g.openhab.core.model.script.batiment] - Cuisine occupée : consigne remontée à 21 °C
2026-10-27 16:41:51.332 [INFO ] [g.openhab.core.model.script.batiment] - Salle A vide : consigne abaissée à 18 °C
2026-10-27 16:42:41.443 [INFO ] [g.openhab.core.model.script.batiment] - Salle A occupée : consigne remontée à 21 °C
2026-10-27 16:43:31.561 [INFO ] [g.openhab.core.model.script.batiment] - Salle A vide : consigne abaissée à 18 °C
2026-10-27 16:45:12.446 [INFO ] [g.openhab.core.model.script.batiment] - Cuisine vide : consigne abaissée à 18 °C
PS C:\Users\Administrator\Desktop\UDS\Automne\IFT 605  IFT 713 - Systèmes répartis et multiagents\Projets\Projet1\smartagent-iot>
```

FIGURE 6 – Journal d'openHAB : règles « Économie d'énergie » / « Confort » et « Équilibrage signalé » (à 16 :41 :24, écart de 4,1 °C ; voir la section 7).

6 Java RMI : agents et registre

Interface distante. Chaque agent implémente `IAgentPiece`, qui étend `Remote` ; chaque méthode déclare `RemoteException` et n'utilise que des types sérialisables (primitifs et `String`).

```
public interface IAgentPiece extends Remote {
    double  getTemperature()                 throws RemoteException;
    int     getLuminosite()                  throws RemoteException;
    boolean isOccupee()                      throws RemoteException;
    void    setConsigneTemperature(double t) throws RemoteException; // publie sur MQTT
    String  getEtatJson()                    throws RemoteException;
    boolean isEnTimeout()                    throws RemoteException; // ajout : détection de panne
    String  consulterVoisin(String piece)    throws RemoteException; // appel d'agent à agent
}
```

Deux ajouts dépassent l'interface minimale : `isEnTimeout()`, indispensable pour que le coordinateur détecte un capteur muet, et `consulterVoisin()`, décrit plus bas. Ajouter une méthode à une interface distante oblige tous les implémenteurs à la fournir, y compris le faux agent de test.

Registre et exportation. Le registre tourne sur le port **2199** dans sa propre JVM (`RegistreRmi`). Chaque agent étend `UnicastRemoteObject`, dont le constructeur « exporte » l'objet (ouvre un port TCP pour les appels entrants), puis s'inscrit par `rebind` (qui écrase une ancienne inscription, ce qui permet de relancer un agent après une panne). Trois précautions pratiques : `java.rmi.server.hostname=127.0.0.1` (sinon le stub peut contenir une adresse IP inatteignable, par exemple celle de Docker ou d'un VPN) ; une référence statique à l'objet exporté (sinon le ramasse-miettes peut le collecter) ; et la création du registre seulement s'il n'existe pas déjà.

Thread-safety. Dans un agent, le thread du callback MQTT *écrit* les mesures pendant que les threads RMI les *lisent*. Les champs de `EtatPiece` sont donc `volatile` (visibilité immédiate entre threads). Cela suffit car chaque champ est indépendant des autres ; en revanche, `getEtatJson()` peut mélanger des valeurs issues de

deux messages successifs, limite acceptée et documentée.

```
private volatile double  temperature = Double.NaN;
private volatile int     luminosite;
private volatile boolean occupee;
private volatile long    derniereMaj = -1;            // -1 : aucun message reçu

public boolean enTimeout() {
    long derniere = derniereMaj;
    return derniere < 0 || horloge.getAsLong() - derniere > timeoutMs;  // 30 000 ms
}
```

Partage d'état entre agents. La méthode `consulterVoisin(piece)` permet à un agent d'interroger *directement* l'agent d'une autre pièce, sans passer par le coordinateur : il retrouve le voisin dans le registre puis appelle son `getEtatJson()`. Si le voisin est arrêté ou absent, l'appelant reçoit une `RemoteException` : la panne d'un voisin n'arrête jamais l'agent.

```
public String consulterVoisin(String piece) throws RemoteException {
    if (nom.equals(piece)) { return getEtatJson(); }
    try {
        Registry registre = LocateRegistry.getRegistry(Constantes.HOTE_RMI, Constantes.PORT_RMI);
        IAgentPiece voisin = (IAgentPiece) registre.lookup(Constantes.nomAgent(piece));
        return voisin.getEtatJson();                        // appel RMI d'agent à agent
    } catch (NotBoundException e) {
        throw new RemoteException("Agent introuvable dans le registre : " + piece, e);
    } catch (RemoteException e) {
        throw new RemoteException("Voisin injoignable : " + piece, e);
    }
}
```

La figure 7 montre le test : un script `jshell` appelle `consulterVoisin("salleB")` sur l'agent de la salle A ; l'état de la salle B revient en JSON, et le journal de la salle A garde la trace de la consultation. La figure 8 montre le petit client RMI de validation (`ClientRmi`) qui lit l'état d'une pièce puis envoie une consigne.

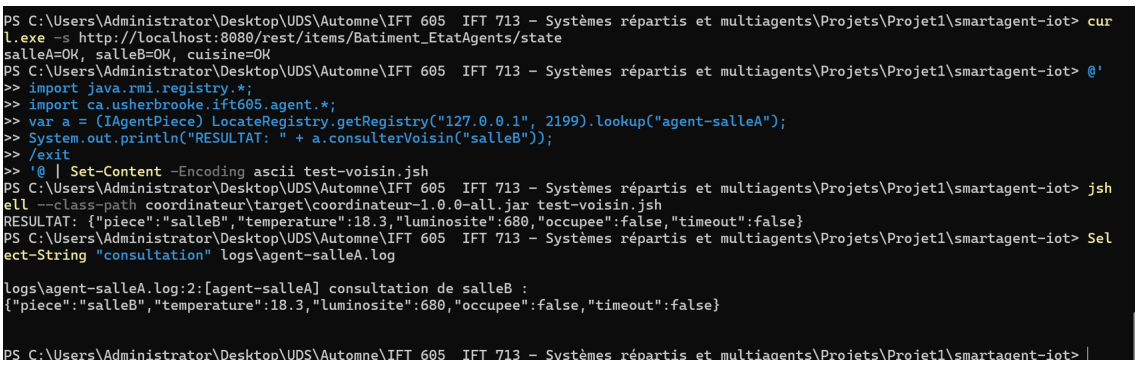


FIGURE 7 – Appel d'agent à agent : salleA consulte salleB par RMI (réponse JSON, trace dans le journal de salleA).

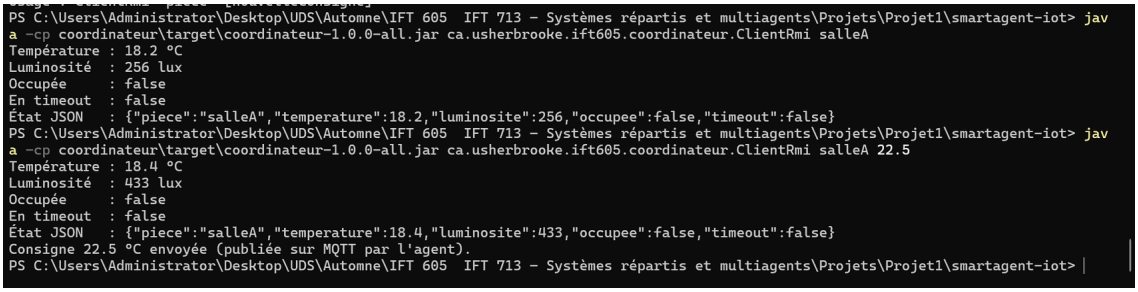


FIGURE 8 – `ClientRmi` : lecture de l'état de salleA, puis envoi de la consigne 22,5 °C (publiée sur MQTT par l'agent).

7 Coordinateur : politique d'équilibrage

Politique. Pour chaque paire adjacente (salleA–salleB et salleB–cuisine), si l'écart absolu de température **dépasse strictement** 4 °C, le coordinateur calcule une consigne égale à la **moyenne** des deux températures (arrondie au dixième) et l'envoie, par RMI, à la pièce **la plus chaude**. À 4,0 °C exactement, rien n'est envoyé

(seuil strict). Un cycle s'exécute toutes les 5 secondes ; il lit *une fois* chaque agent, traite les deux paires, puis publie l'état des agents dans openHAB.

```
static double calculerConsigne(double tChaude, double tFroide) {
    return Math.round((tChaude + tFroide) / 2.0 * 10) / 10.0;      // moyenne au dixième
}
...
double ecart = Math.abs(a.temperature() - b.temperature());
if (ecart <= SEUIL_ECART) { log("INFO", "... équilibré"); return; }
Releve chaude = a.temperature() > b.temperature() ? a : b;
cible.setConsigneTemperature(calculerConsigne(chaude.temperature(), froide.temperature())); // RMI
```

Exemple réel. Le 27 octobre, à 16 :41 :24 (UTC), le coordinateur a observé la cuisine à 21,6 °C et la salle B à 17,5 °C, soit un écart de 4,1 °C $>$ 4,0 °C, et envoyé la consigne $(21{,}6+17{,}5)/2 = 19{,}6$ à la cuisine. La figure 9 détaille la chaîne ; openHAB l'a signalé par la règle « Équilibrage signalé » (figure 6).

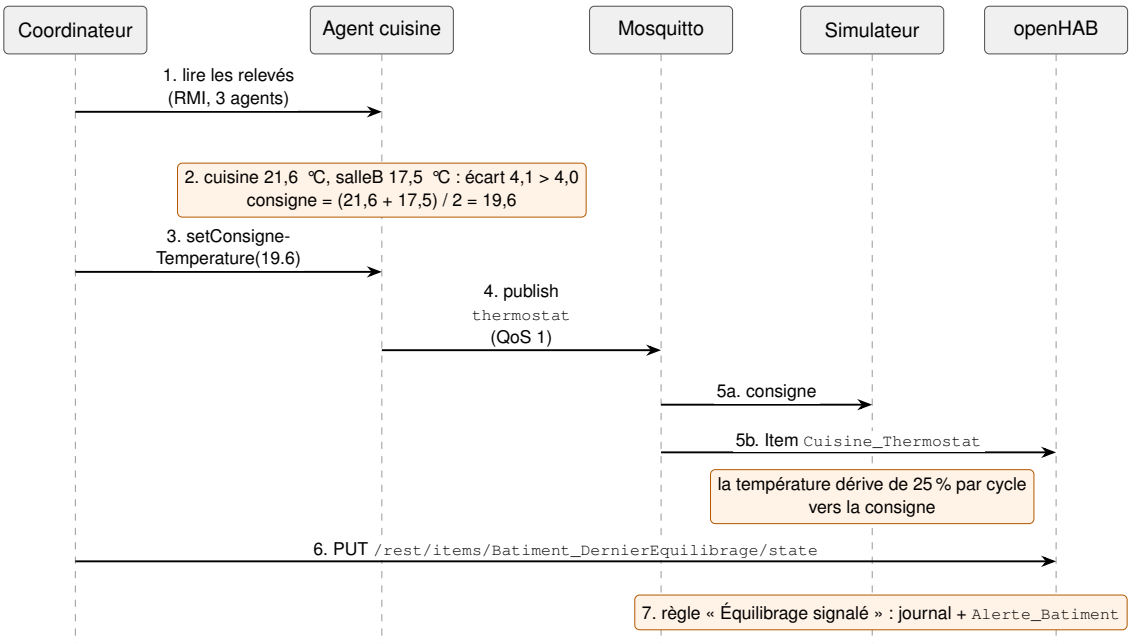


FIGURE 9 – Diagramme de séquence d'un rééquilibrage (écart supérieur à 4 °C).

Intégration avec openHAB. Le coordinateur appelle l'API REST d'openHAB (`PUT /rest/items/<item>/state`) pour mettre à jour `Batiment_EtatAgents` (statut de chaque agent) et `Batiment_DernierEquilibrage`. Dans notre installation, aucun jeton n'a été nécessaire ; une variable d'environnement `OPENHAB_TOKEN` est prévue si l'API répond 401. Si openHAB est indisponible, le coordinateur le journalise une fois et continue sans lui.

Planification. Le cycle est lancé par `scheduleWithFixedDelay`. Piège classique : une exception non attrapée dans la tâche annulerait *silencieusement* toutes les exécutions suivantes ; la tâche est donc enveloppée dans un `catch (Throwable)`.

```
planificateur.scheduleWithFixedDelay(() -> {
    try { coordinateur.equilibrerTemperatures(); }
    catch (Throwable t) { t.printStackTrace(); }       // sans cela, un seul échec tuerait le planning
}, 0, 5, TimeUnit.SECONDS);
```

8 Tolérance aux pannes

Deux types de pannes sont gérés, avec des mécanismes différents. Avant toute panne, l'état nominal est vérifié : douze processus Java, aucun fichier d'erreur non vide, un journal du coordinateur régulier et un état REST `salleA=OK, salleB=OK, cuisine=OK` (figure 10).

8.1 Scénario 1 : panne d'un capteur (timeout de 30 s)

On coupe les publications de la salle B : `mosquitto_pub -t batiment/salleB/simulateur/panne -m true`.

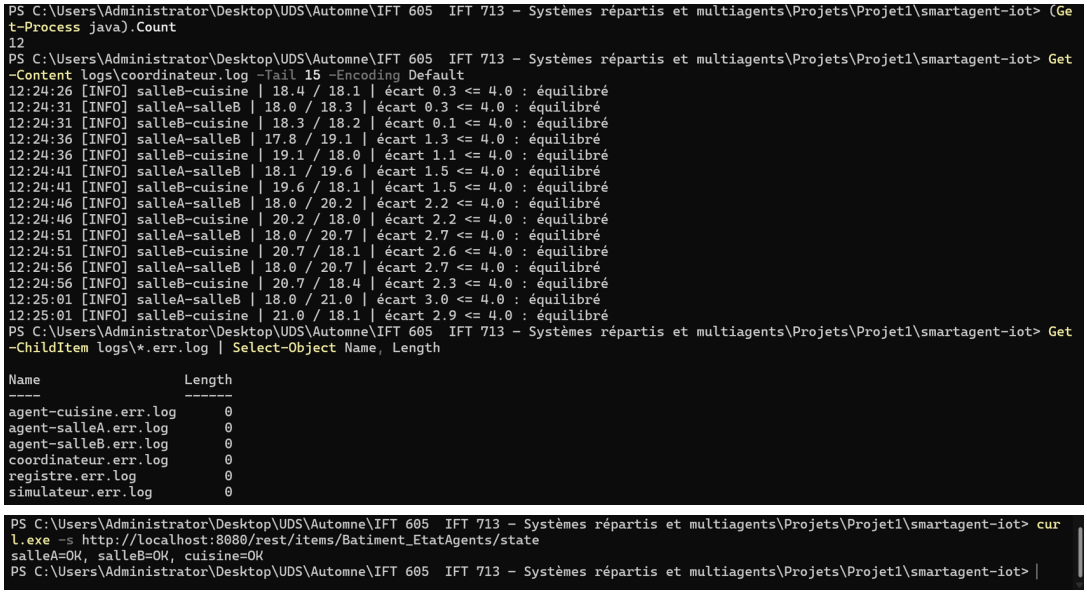


FIGURE 10 – État nominal : 12 processus Java, cycles d'équilibrage du coordinateur, fichiers d'erreurs vides (`Length = 0`) ; puis lecture de l'Item `Batiment_EtatAgents` par l'API REST d'openHAB.

— **Agent** : après plus de 30 s sans message, `isEnTimeout()` renvoie `true`.
— **Coordinateur** : il continue d'utiliser la *dernière valeur connue* de la salle B, avec un avertissement (« capteur en timeout, dernière valeur connue utilisée »), et publie `salleB=TIMEOUT` dans openHAB ; la paire qui ne concerne pas la salle B n'est pas affectée.
— **openHAB** : l'Item de température de la salle B passe à `UNDEF` (fonction *expire*), ce qui déclenche la règle « Capteur hors ligne » (`Capteur hors ligne : SalleB_Temperature`, à 16 :54 :18 UTC, soit environ 30 s après la coupure).

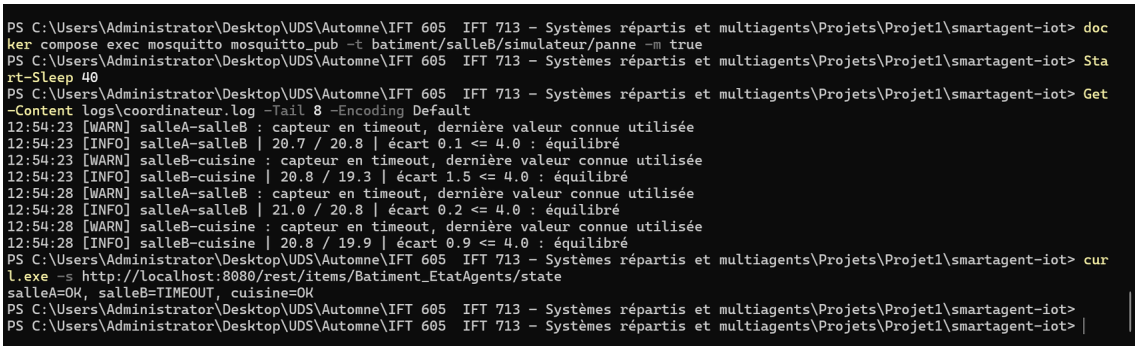


FIGURE 11 – Panne du capteur de la salle B : avertissements du coordinateur et état REST `salleB=TIMEOUT`.

Reprise. Avec `panne=false`, les publications reprennent : 15 s plus tard, l'état REST redevient `salleA=OK, salleB=OK, cuisine=OK` et `SalleB_Temperature` affiche de nouveau une valeur numérique (20,4 °C), *sans redémarrer aucun programme* (figure 12).

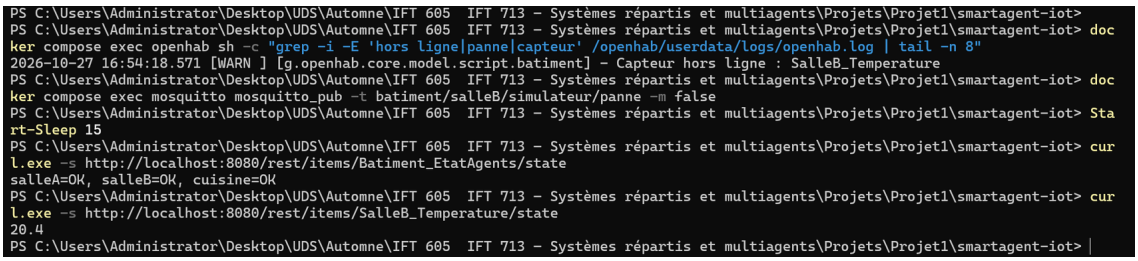


FIGURE 12 – Alerte « capteur hors ligne » dans openHAB, puis reprise après `panne=false`.

8.2 Scénario 2 : arrêt d'un agent (RemoteException)

On arrête la JVM de l'agent de la cuisine (`taskkill /T`). Au cycle suivant, l'appel RMI échoue (`Connection refused`). Le coordinateur capture la `RemoteException` *dans un bloc dédié à cet agent*, journalise l'erreur, marque `cuisine=HORS_LIGNE`, oublie le stub périmé et **ignore uniquement la paire concernée** : la paire salleA–salleB continue d'être équilibrée toutes les 5 secondes (figure 13). **Reprise.** L'agent est relancé à la

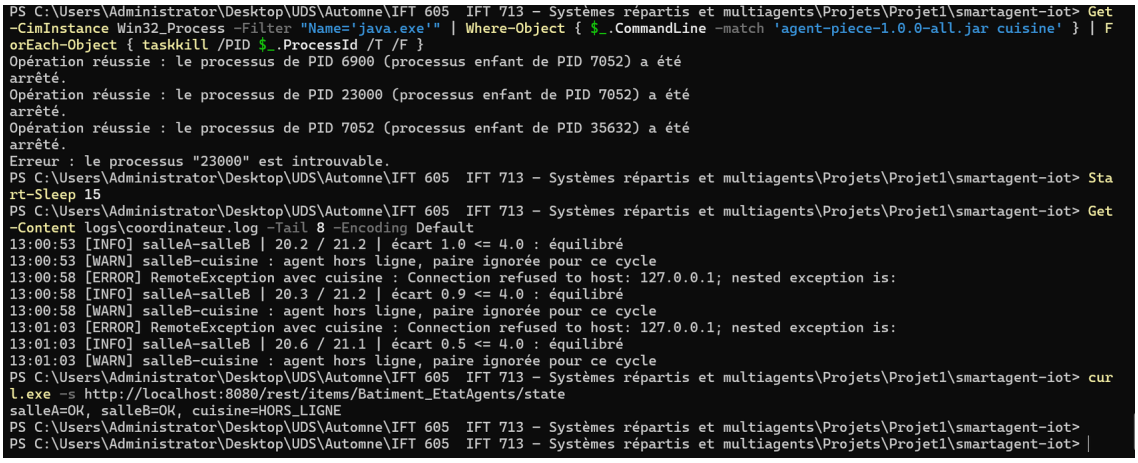


FIGURE 13 – Arrêt de l'agent de la cuisine : `RemoteException`, paire ignorée, `cuisine=HORS_LIGNE`.

main ; il s'inscrit de nouveau avec `rebind`. Comme la recherche dans le registre est *paresseuse* (faite au premier besoin) et que le stub périmé a été oublié, le coordinateur retrouve l'agent **sans redémarrage** : dans les 20 s, la paire salleB–cuisine est de nouveau équilibrée et l'état redevient `salleA=OK, salleB=OK, cuisine=OK` (figure 14).

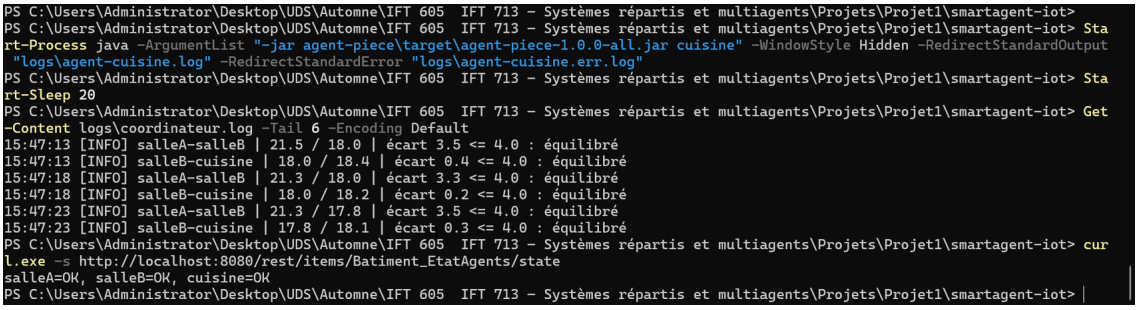


FIGURE 14 – Relance de l'agent de la cuisine : le coordinateur le retrouve et l'état redevient complet.

9 Tests et validation

Tests unitaires (JUnit 5, 15 tests, tous verts). Ils portent sur la logique pure, sans réseau : le coordinateur est testé avec de faux agents en mémoire (`AgentFactice`), l'état d'une pièce avec une horloge injectée (pas besoin d'attendre 30 s).

Validation sur le système réel. Les comportements qui traversent le réseau ne sont pas couverts par JUnit ; ils ont été vérifiés à la main, avec les preuves citées ci-dessus : flux MQTT jusqu'au tableau de bord (figure 5), Things en ligne (figure 3), règles (figure 6), appel d'agent à agent et `ClientRmi` (figures 7 et 8), rééquilibrage (figure 9), deux scénarios de panne avec reprise (figures 11 à 14).

10 Analyse critique

1. Conflit entre règles openHAB et coordinateur (observé). Deux acteurs écrivent la même consigne. Après la consigne de 19,6 °C envoyée à la cuisine à 16 :41 :24, la règle « Cuisine vide : consigne abaissée à 18 °C » l'a écrasée à 16 :41 :32, car le simulateur avait fait basculer l'occupation. Les règles d'occupation et la politique d'équilibrage n'ont pas d'arbitrage. Amélioration : un seul « écrivain » du thermostat (le coordinateur, qui connaîtrait aussi l'occupation), ou des priorités explicites.

Classe de test	Tests	Comportements vérifiés
`EtatPieceTest`	5	application des trois types de mesures ; rejet des valeurs invalides en gardant l'ancienne ; timeout sans message ; timeout après plus de 30 s ; contenu du JSON.
`CoordinateurBatimentTest`	9	consigne = moyenne arrondie ; écart > 4 envoie la moyenne à la pièce la plus chaude ; la plus froide ne reçoit rien ; écart $= 4$ ne déclenche rien ; panne d'une salle (les autres paires continuent) ; agent introuvable sans plantage ; timeout avec dernière valeur connue ; agent sans mesure ignoré ; retour à la normale après panne.
`CapteurSimulateurPlagesTest`	1	température bornée aux plages documentées (15 à 30 °C).

TABLE 4 – Couverture des tests JUnit.

2. Politique d'équilibrage naïve. Pas d'hystérésis (une température qui oscille autour de 4 °C peut produire des consignes à répétition), seule la pièce la plus chaude est commandée, l'occupation n'est pas prise en compte, et la consigne moyenne n'est qu'une heuristique.

3. Limites de RMI. Couplage fort à Java, traversée de pare-feu délicate (ports d'objets exportés), pas de sécurité (désérialisation), `java.rmi.server.hostname` à régler. Le registre est un point unique de défaillance *pour les nouvelles recherches* : les stubs déjà obtenus continuent de fonctionner s'il tombe, mais un agent relancé ne pourrait plus s'inscrire. Le coordinateur est lui-même un point unique : s'il tombe, plus aucun équilibrage.

4. Limites de MQTT. Broker anonyme et sans TLS (acceptable en test, pas en production), `persistence false` (les messages ne survivent pas à un redémarrage du broker), QoS 0 qui autorise des pertes de mesures.

5. Concurrence. `volatile` suffit pour des champs indépendants ; s'il fallait garantir un invariant *entre* plusieurs champs (par exemple une température et son horodatage), il faudrait `synchronized` ou un objet immuable remplacé atomiquement. La détection de timeout existe à deux endroits (agent et `expire` d'openHAB) : ces deux mécanismes se complètent, mais le délai de 30 s est dupliqué dans le code et dans les Items.

6. Passage à l'échelle. N pièces donnent N JVM et un coordinateur qui interroge chaque agent à chaque cycle (modèle *pull*) ; un modèle *push* ou des événements MQTT passeraient mieux à l'échelle.

7. Limites de notre validation. Nous le disons explicitement : (a) la règle « Alerte surchauffe » (plus de 28 °C) n'a pas été déclenchée pendant nos essais ; elle peut l'être en forçant la température à 29 °C par le topic `simulateur/temperature` ; (b) le passage à `UNDEF` est démontré par la règle « Capteur hors ligne » dans le journal, mais nous n'avons pas de capture du tableau de bord affichant « - » au moment de la panne ; (c) `consulterVoisin` n'est pas couvert par JUnit (le faux agent renvoie une valeur fixe) ; (d) les tests réseau (MQTT, RMI, REST) sont manuels, pas automatisés.

11 Difficultés rencontrées

Le tableau 5 résume les principaux obstacles rencontrés, leurs causes et les solutions retenues.

La leçon la plus utile concerne le broker de l'hôte : quand un système réparti « marche à moitié », il faut vérifier *qui écoute réellement* sur chaque port avant de soupçonner le code.

12 Conclusion

La plateforme remplit son rôle de bout en bout : les capteurs simulés alimentent le broker, les agents exposent l'état de chaque pièce par RMI, le coordinateur rééquilibre les températures et informe openHAB, et le système survit aux deux types de pannes étudiés puis reprend sans redémarrage. Le tableau 6 résume les critères du barème et les preuves fournies.

Pistes d'amélioration. Arbitrer les écritures sur le thermostat, ajouter une hystérésis à la politique d'équilibrage, authentifier et chiffrer MQTT (TLS), automatiser des tests d'intégration réels (RMI et MQTT) et ajouter un délai de réponse RMI pour qu'un agent *gelé* (et non arrêté) ne bloque pas le cycle.

Problème	Cause	Solution
Docker ne démarre pas (« Virtual Machine Platform not enabled »)	Fonctionnalité Windows désactivée.	Activer la fonctionnalité, `wsl --update`, redémarrer.
`mvn` introuvable	Maven décompressé mais absent du `PATH`.	Ajouter `C:\apache-maven-3.10.0\bin` au `PATH`, rouvrir le terminal.
Scripts `.sh` et `tail` inopérants	PowerShell n'exécute pas bash.	Bloc PowerShell (`Start-Process`), `Get-Content -Wait`, `taskkill /T`.
openHAB affiche « – » alors que les agents reçoivent des mesures	Un **Mosquitto installé sur Windows** écoutait aussi sur 1883 : les programmes Java parlaient à ce broker, openHAB (dans Docker) à celui du conteneur. Deux brokers, deux mondes.	Diagnostic avec `Get-NetTCPConnection -LocalPort 1883` puis désinstallation du broker de l'hôte ; un seul écouteur reste.
Énoncé ambigu	`getEtat` / `getEtatJson` ; syntaxe du Bridge MQTT d'openHAB 4.	Nom `getEtatJson` retenu ; Bridge + Things en fichiers (syntaxe 4.x).

TABLE 5 – Principales difficultés, causes et solutions.

Critère	Preuves dans ce rapport
MQTT (pub/sub)	Topics et QoS (tab. 2), mesures visibles au tableau de bord (fig. 5).
openHAB Things	Quatre Things en ligne (fig. 3), Items liés, sitemap.
openHAB Rules	Neuf règles (tab. 3), exécution prouvée par le journal (fig. 6).
Java RMI	Interface, registre 2199, trois agents (§6), `ClientRmi` et `consulterVoisin` (fig. 7, 8).
Coordinateur	Politique stricte à 4 °C, exemple réel 4,1 °C → 19,6 °C (§7, fig. 9).
Tolérance aux pannes	Timeout, `UNDEF`, `RemoteException`, reprises (fig. 11 à 14).
Rapport	Architecture (fig. 1), diagramme de séquence, analyse critique (§10).

TABLE 6 – Bilan par rapport aux critères d'évaluation.

A Reproduire le système (PowerShell)

Depuis le dossier `smartagent-iot` :

```
docker compose up -d                        # Mosquitto + openHAB
mvn clean package                           # compile, 15 tests, jars *-all.jar

New-Item -ItemType Directory -Force logs | Out-Null
function Lancer($nom, $arguments) {
  Start-Process java -ArgumentList $arguments -WindowStyle Hidden -RedirectStandardOutput "logs\$nom.log" -
     RedirectStandardError "logs\$nom.err.log"
}
Lancer "registre" "-cp agent-piece\target\agent-piece-1.0.0-all.jar ca.usherbrooke.ift605.agent.RegistreRmi"
Start-Sleep 2
foreach ($p in "salleA","salleB","cuisine") { Lancer "agent-$p" "-jar agent-piece\target\agent-piece-1.0.0-all.jar
     $p" }
Start-Sleep 2
Lancer "simulateur"   "-jar sensor-simulator\target\sensor-simulator-1.0.0-all.jar"
Start-Sleep 1
Lancer "coordinateur" "-jar coordinateur\target\coordinateur-1.0.0-all.jar"
```

Tableau de bord : `http://localhost:8080/basicui/app?sitemap=batiment`. Pannes simulées :

```
docker compose exec mosquitto mosquitto_pub -t batiment/salleB/simulateur/panne -m true    # coupe salleB
docker compose exec mosquitto mosquitto_pub -t batiment/salleB/simulateur/panne -m false   # reprise
java -cp coordinateur\target\coordinateur-1.0.0-all.jar ca.usherbrooke.ift605.coordinateur.ClientRmi salleA 22.5
```